

■ Lab 5 — Outputs & Variables

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Exporté des attributs de ressources avec les **outputs**
- Déclaré et utilisé des **variables** avec différents types
- Assigné des variables via **CLI**, **tfvars** et **variables d'environnement**
- Ajouté des **règles de validation** sur les variables

■ Étape 1 — Préparer le dossier

```
cd labs/lab5-outputs
```

```
lab5-outputs/  
■■■ versions.tf  
■■■ variables.tf  
■■■ main.tf  
■■■ outputs.tf  
■■■ terraform.tfvars
```

■ Étape 2 — versions.tf avec backend S3

versions.tf

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }

  backend "s3" {
    region      = "eu-west-1"
    use_lockfile = true
    encrypt     = true
  }
}

provider "aws" {
  region = var.region
}
```

■ Étape 3 — Déclarer les variables

`variables.tf`

```
variable "username" {
  description = "Votre prenom - permet de differencier les ressources entre participants"
  type        = string

  validation {
    condition     = length(var.username) >= 2 && length(var.username) <= 20
    error_message = "Le username doit contenir entre 2 et 20 caractères."
  }
}

variable "region" {
  description = "Region AWS de déploiement"
  type        = string
  default     = "eu-west-1"
}

variable "instance_type" {
  description = "Type d'instance EC2"
  type        = string
  default     = "t2.micro"

  validation {
    condition     = contains(["t2.micro", "t2.small", "t2.medium"], var.instance_type)
    error_message = "Le type d'instance doit être t2.micro, t2.small ou t2.medium."
  }
}

variable "environment" {
  description = "Environnement de déploiement"
  type        = string

  default     = "dev"

  validation {
    condition     = contains(["dev", "prod"], var.environment)
    error_message = "L'environnement doit être dev ou prod."
  }
}
```

■ Le bloc `validation` permet de définir des règles personnalisées sur les variables. Si la condition est `false`, Terraform affiche le `error_message` et stoppe.

■ Étape 4 — main.tf

`main.tf`

```
resource "aws_security_group" "lab5_sg" {
  name          = "lab5-sg-${var.username}-${var.environment}"
  description   = "Security group Lab 5 - ${var.username}"

  tags = {
    Name       = "lab5-sg-${var.username}"
    Lab        = "lab5"
    Username   = var.username
    Environment = var.environment
  }
}

resource "aws_instance" "lab5_instance" {
  ami              = "ami-0694d931cee176e7d"
  instance_type    = var.instance_type
  vpc_security_group_ids = [aws_security_group.lab5_sg.id]

  tags = {
    Name       = "lab5-instance-${var.username}"
    Lab        = "lab5"
    Username   = var.username
    Environment = var.environment
  }
}
```

■ Étape 5 — Outputs

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance EC2"
  value      = aws_instance.lab5_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value      = aws_instance.lab5_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value      = aws_security_group.lab5_sg.id
}

output "security_group_name" {
  description = "Nom du Security Group"
  value      = aws_security_group.lab5_sg.name
}

output "deployment_summary" {
  description = "Résumé du déploiement"
  value = {
    username    = var.username
    environment = var.environment
    instance    = aws_instance.lab5_instance.id
    region      = var.region
  }
}
```

■ *Les outputs peuvent exposer :*

- Un **attribut spécifique** : ``aws_instance.lab5_instance.id``

- **Tous les attributs** : *via un objet comme ``deployment_summary``*

- *Dans un module, les outputs sont accessibles par d'autres ressources*

■ Étape 6 — Initialiser avec le Backend S3

```
BUCKET_NAME=$(cat $HOME/tf-training-info.txt | awk '{print $NF}')

terraform init \
  -backend-config="bucket=${BUCKET_NAME}" \
  -backend-config="key=terraform.tfstate"
```

■ Étape 7 — Les 3 façons d'assigner les variables

Méthode 1 — CLI flag (`-var`)

```
terraform apply \  
-var="username=<votre-prenom>" \  
-var="environment=dev" \  
-var="instance_type=t2.micro"
```

Méthode 2 — Fichier tfvars

Créez `terraform.tfvars` :

```
username      = "<votre-prenom>"  
environment   = "dev"  
instance_type = "t2.micro"
```

Puis appliquez sans `-var` :

```
terraform apply
```

■ Terraform charge automatiquement `terraform.tfvars` s'il est présent dans le dossier. Pour un autre fichier :
`terraform apply -var-file="prod.tfvars"`

Méthode 3 — Variables d'environnement (`TF\_VAR\_`)

```
export TF_VAR_username="<votre-prenom>"  
export TF_VAR_environment="dev"  
export TF_VAR_instance_type="t2.micro"  
  
terraform apply
```

■ Le préfixe `TF_VAR_` suivi du nom de la variable est reconnu automatiquement par Terraform.

■ Étape 8 — Tester la validation

Testez une valeur invalide pour voir la validation en action :

```
terraform plan -var="username=<votre-prenom>" -var="environment=staging"
```

Résultat attendu :

```
Error: Invalid value for variable
  L'environnement doit être dev ou prod.
```

```
terraform plan -var="username=<votre-prenom>" -var="instance_type=t3.large"
```

Résultat attendu :

```
Error: Invalid value for variable
  Le type d'instance doit être t2.micro, t2.small ou t2.medium.
```

■ Étape 9 — Utiliser terraform output

Après le `terraform apply` :

```
# Afficher tous les outputs
terraform output

# Afficher un output spécifique
terraform output instance_id
terraform output instance_public_ip

# Afficher un output en format JSON
terraform output -json deployment_summary

# Afficher un output brut (sans guillemets) – utile dans les scripts
terraform output -raw instance_id
```

■ Étape 10 — Nettoyage

```
terraform destroy -var="username=<votre-prenom>"
```

■ Validation du Lab

#	Critère	Validé
1	Variables déclarées avec types et descriptions	■

#	Critère	Validé
2	Validation bloque <code>`environment=staging`</code>	■
3	Validation bloque <code>`instance_type=t3.large`</code>	■
4	<code>`terraform apply`</code> via <code>`-var`</code> fonctionne	■
5	<code>`terraform apply`</code> via <code>`terraform.tfvars`</code> fonctionne	■
6	<code>`terraform apply`</code> via <code>`TF_VAR_`</code> fonctionne	■
7	<code>`terraform output`</code> affiche les 5 outputs	■
8	<code>`terraform output -json deployment_summary`</code> affiche un objet JSON	■
9	<code>`terraform destroy`</code> supprime les ressources	■

■ ■ Résolution des problèmes courants

Problème	Cause	Solution
<code>`Error: No value for required variable`</code>	Variable sans default non fournie	Ajouter <code>`-var="username=..."`</code> ou <code>`terraform.tfvars`</code>
<code>`Error: Invalid value for variable`</code>	Validation échouée	Vérifier la valeur fournie
<code>`TF_VAR_`</code> ignoré	Mauvais nom de variable	Vérifier l'orthographe exacte

■ ****Fin du Lab 5 — Votre code est dynamique grâce aux variables !****

*Le Lab 6 introduit les ****locals**** pour simplifier et centraliser les expressions dans votre code.*